

Day 3

Overview

Dashboard

Dimensionality Reduction
(PCA)

Clustering (K-Means)

Neural Networks

← Back to Day 3

Neural Networks

Building brains with layers of linear maps and nonlinear activations.

Multi-Layer Perceptron (MLP)

Neural Networks are composed of layers of "neurons". A single neuron computes a weighted sum of its inputs, adds a bias, and passes the result through a non-linear activation function.

Mathematical Model of a Neuron

$$y = \phi \left(\sum_{i=1}^n w_i x_i + b \right)$$

Where:

- x_i are the input features.
- w_i are the weights (learnable parameters).
- b is the bias (learnable parameter).
- ϕ is the activation function (e.g., ReLU, Sigmoid).

Common Activation Functions

- **Sigmoid:** $\sigma(z) = \frac{1}{1+e^{-z}}$ (Range: 0 to 1)
- **ReLU (Rectified Linear Unit):** $f(z) = \max(0, z)$ (Range: 0 to ∞)
- **Tanh:** $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$ (Range: -1 to 1)

Interactive Neuron

Parameters

Input x1: 1.0

Weight w1: 0.5

Input x2: -1.0

Weight w2: 0.5

Bias b: 0.0

Activation Function:

ReLU

Visualization

Day 3

Overview

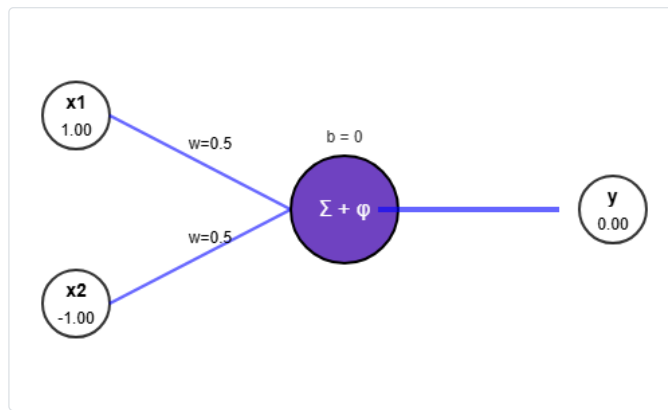
Dashboard

Dimensionality Reduction
(PCA)

Clustering (K-Means)

Neural Networks

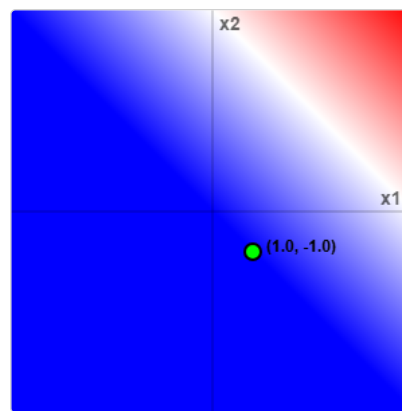
← Back to Day 3



Weighted Sum (z): 0.00

Output (a): 0.0000

Activation Landscape (Output vs x1, x2)



Brightness = Output Value. Green Dot = Current Input.

What's happening?

- **Change x1, x2:** Moves the **Green Dot**. This is your data point.
- **Change w1, w2:** Rotates the landscape. It spins the boundary between Blue (Low) and Red (High).
Why? w1 tilts the plane left/right. w2 tilts it up/down. Changing both spins the angle of the slope.
- **Change Bias (b):** Shifts the "cliff" (transition zone) relative to the center.

Backpropagation: The "Blame Game"

How does the network learn? It compares its prediction to the actual answer (Target) and sends the error **backwards** to adjust the weights.

The Chain Rule (Simplified)

We want to change a weight w to reduce the Error E .

$$w_{new} = w_{old} - \eta \cdot \frac{\partial E}{\partial w}$$

To find $\frac{\partial E}{\partial w}$ (the gradient), we trace the path backwards:

1. **Error Term:** "The image is a 'cat', so the Target is 1.0. We predicted 0.2. This large gap (Error) is sent backwards to correct the mistake."
2. **Derivative of Activation:** "Are we in the sensitive range of the neuron? (If the neuron is saturated, learning is slow)."
3. **Input Value:** "Was this input active? If yes, it contributed to the mistake and its weight must change."

Day 3

Overview

Dashboard

Dimensionality Reduction
(PCA)

Clustering (K-Means)

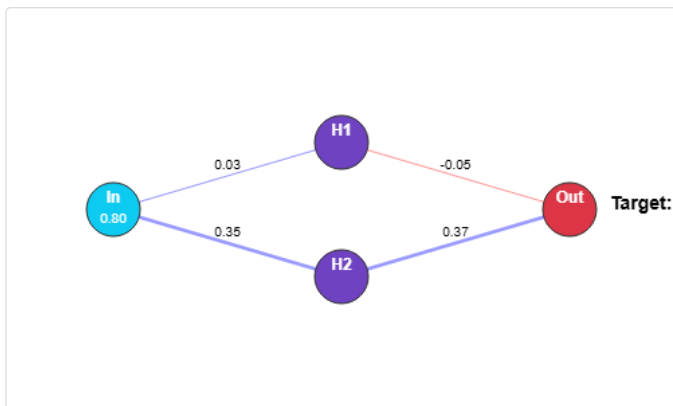
Neural Networks

← Back to Day 3

"If the input was high, and the error is high, this weight is highly responsible."

Visualizing Gradient Flow (Digits Example)

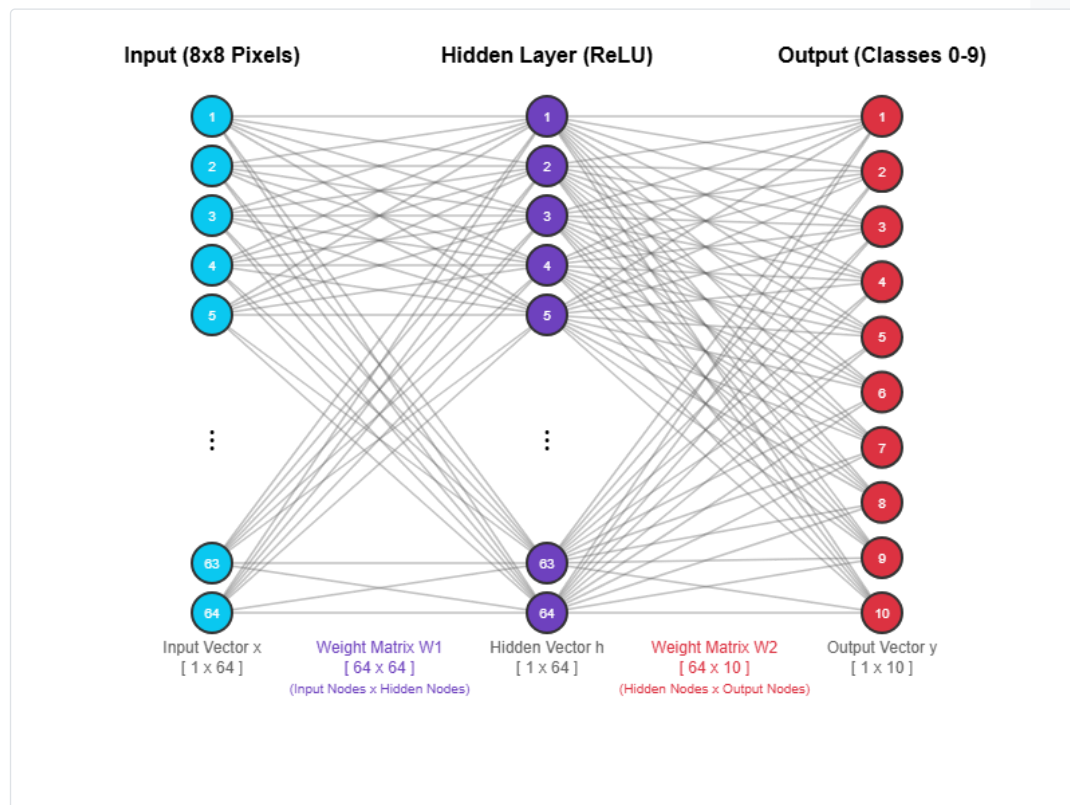
Simplified: One Pixel → 2 Hidden Neurons → Output "Is it a cat?"



Weights reset. Ready to train...

Network Architecture Visualization

Visualizing the MLP architecture used in the code below (64 Inputs → 64 Hidden → 10 Outputs).



Python Implementation (scikit-learn)

Training a Multi-Layer Perceptron (MLP) on the digits dataset.

```
from sklearn.neural_network import MLPClassifier
```

Day 3

Overview

Dashboard

Dimensionality Reduction
(PCA)

Clustering (K-Means)

Neural Networks

← Back to Day 3

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import load_digits
from sklearn.metrics import accuracy_score, classification_report,
ConfusionMatrixDisplay
import matplotlib.pyplot as plt

def simple_mlp_demo():
    # Load data
    X, y = load_digits(return_X_y=True)

    # Visualize Dataset: Show samples of each digit (0-9)
    # We display them in a single row for a quick overview.
    print("Visualizing one sample per digit...")
    fig, axes = plt.subplots(1, 10, figsize=(12, 2))
    for i in range(10):
        # Find the first sample of digit i
        sample_img = X[y == i][0].reshape(8, 8)
        axes[i].imshow(sample_img, cmap='gray')
        axes[i].set_title(f"{i}")
        axes[i].axis('off')
    plt.tight_layout()
    plt.show()

    # Split and Scale (Scaling is crucial for Neural Networks!)
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y
    )
    scaler = StandardScaler()
    X_train_s = scaler.fit_transform(X_train)
    X_test_s = scaler.transform(X_test)

    # Define MLP: 1 hidden layer with 64 neurons
    mlp = MLPClassifier(
        hidden_layer_sizes=(64,),
        activation="relu",
        solver="adam",
        max_iter=300,
        random_state=1
    )

    # Train
    mlp.fit(X_train_s, y_train)

    # Evaluate
    y_pred = mlp.predict(X_test_s)
    print("MLP accuracy on digits:", accuracy_score(y_test, y_pred))

    # Detailed classification report
    print("\nClassification Report:\n", classification_report(y_test, y_pred))

    # Visualization of Confusion Matrix
    # (Requires matplotlib to be installed and a display environment)
    ConfusionMatrixDisplay.from_predictions(y_test, y_pred)
    plt.title("Confusion Matrix")
    plt.show()
simple_mlp_demo()
```

Day 3

[Overview](#)

[Dashboard](#)

[Dimensionality Reduction
\(PCA\)](#)

[Clustering \(K-Means\)](#)

[Neural Networks](#)

[← Back to Day 3](#)